Laboratorio de Túneles SSH

Prácticas Guiadas - Entorno Educativo

Versión: 1.0
Duración estimada: 4-6 horas
Nivel: Intermedio
Entorno: 10 máquinas Kali + 1 servidor Ubuntu compartido

📋 Índice

1. Información General
2. Preparación del Entorno
3. Práctica 1: Introducción a SSH y Autenticación
4. Práctica 2: Local Port Forwarding (-L)
5. Práctica 3: Dynamic Port Forwarding (-D) - SOCKS Proxy
6. Práctica 4: Remote Port Forwarding (-R)
7. Práctica 5: Túneles Múltiples Simultáneos
8. Práctica 6: Escenario Real - Acceso a Red Interna
9. Práctica 7: Seguridad y Auditoría de Túneles
10. Anexo A: Comandos de Referencia
11. Anexo B: Solución de Problemas

---

1. Información General

1.1 Objetivos de Aprendizaje

Al finalizar estas prácticas, el alumno será capaz de:

- ✅ Configurar y utilizar los 3 tipos de túneles SSH
- ✅ Identificar casos de uso apropiados para cada tipo de túnel
- ✅ Diagnosticar problemas comunes de conectividad
- ✅ Implementar soluciones de acceso remoto seguro
- ✅ Auditar y monitorizar túneles activos

1.2 Entorno de Laboratorio

Rol	Sistema	Cantidad	Credenciales
Cliente	Kali Linux	10 (una por alumno)	Usuario: `kali`
Servidor	Ubuntu Server	1 (compartido)	`student01`..`student10` / `lab2025`

1.3 Topología de Red

```
[Kali Alumno 01] ----\
[Kali Alumno 02] -----|
...                   |--> [Ubuntu Server]
[Kali Alumno 10] -----/      192.168.99.50
       ↓                          ↓
 localhost:9090              Servicios internos
 localhost:9092              Puerto 8080 (web demo)
 localhost:3000              Puertos 9091-9100 (-R)
```

---

2. Preparación del Entorno

2.1 Verificación Previa (5 minutos)

En tu máquina Kali, ejecuta:

```
# 1. Verifica conectividad con el servidor
ping -c 3 192.168.99.50

# 2. Verifica que el puerto SSH está accesible
nc -zv 192.168.99.50 22

# 3. Descarga el script de laboratorio
cd ~
wget http://192.168.99.50/ssh_lab.sh -O ssh_lab.sh 2>/dev/null || \
  curl -O http://192.168.99.50/ssh_lab.sh

# 4. Hazlo ejecutable
chmod +x ssh_lab.sh

# 5. Edita tu configuración
nano ssh_lab.sh
# Modifica:
#   STUDENT_ID="XX"        (tu número: 01 a 10)
#   UBUNTU_SERVER_IP="192.168.99.50"

# 6. Ejecuta diagnóstico
./ssh_lab.sh check
```

✅ Checklist de verificación:

- ☐ Ping al servidor funciona
- ☐ Puerto 22 accesible
- ☐ Script descargado y ejecutable
- ☐ Diagnóstico pasa sin errores críticos

---

3. Práctica 1: Introducción a SSH y Autenticación

Duración: 20 minutos
Dificultad: ⭐ Básica

3.1 Objetivos

- Comprender el funcionamiento básico de SSH
- Configurar autenticación por claves públicas
- Verificar la identidad del servidor

3.2 Ejercicios

Ejercicio 1.1: Primera Conexión

```
# Conéctate al servidor por primera vez
ssh studentXX@192.168.99.50
# Introduce password: lab2025

# Observa el mensaje de "fingerprint"
# ¿Qué información proporciona?
```

```
# Respuesta: ____________________________________________

# Sal de la sesión
exit
```

Ejercicio 1.2: Generación de Claves

```
# Genera un par de claves SSH (si no existe)
ssh-keygen -t ed25519 -f ~/.ssh/id_rsa_lab -C "alumno_XX"

# Verifica los archivos creados
ls -la ~/.ssh/

# ¿Qué permisos tienen? Deberían ser:
# id_rsa_lab:     600 (rw-------)
# id_rsa_lab.pub: 644 (rw-r--r--)
```

Ejercicio 1.3: Copia de Clave Pública

```
# Copia tu clave pública al servidor
ssh-copy-id -i ~/.ssh/id_rsa_lab.pub studentXX@192.168.99.50

# Verifica que se añadió correctamente
ssh studentXX@192.168.99.50 "cat ~/.ssh/authorized_keys"

# Ahora conecta SIN password
ssh -i ~/.ssh/id_rsa_lab studentXX@192.168.99.50
```

3.3 Verificación

```
# Ejecuta el script automatizado
./ssh_lab.sh all

# Deberías ver:
# ✓ Conexión SSH funcional
# ✓ Servicio web disponible
# ✅ Túneles establecidos
```

3.4 Preguntas de Reflexión

1. ¿Por qué es más seguro usar claves que passwords?
   Respuesta: **********_____**********

2. ¿Qué es el "fingerprint" de un servidor SSH?
   Respuesta: **********_____**********

3. ¿Qué ocurre si pierdes tu clave privada?
   Respuesta: **********_____**********

---

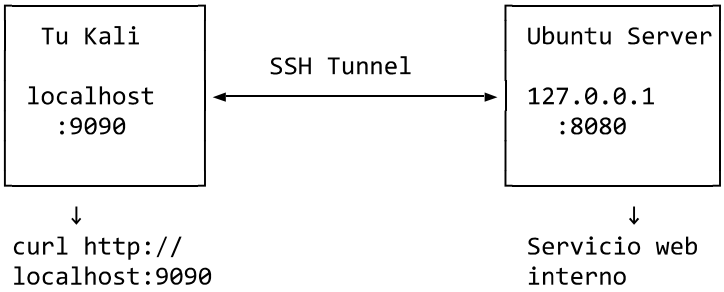
4. Práctica 2: Local Port Forwarding (-L)

Duración: 45 minutos
Dificultad: ⭐⭐ Intermedia

4.1 Objetivos

- Entender el concepto de Local Port Forwarding
- Acceder a servicios remotos como si fueran locales
- Configurar redirección de puertos

4.2 Teoría: ¿Qué es -L?



Comando:

```
ssh -L [puerto_local]:[destino_desde_servidor]:[puerto_destino] usuario@servidor
```

4.3 Ejercicios

Ejercicio 2.1: Túnel Básico

```
# Establece el túnel manual (sin el script)
ssh -i ~/.ssh/id_rsa_lab \
    -L 9090:localhost:8080 \
   -N -f studentXX@192.168.99.50

# Explica cada parámetro:
# -i: ____________________________________________
# -L: ____________________________________________
# -N: ____________________________________________
# -f: ____________________________________________
```

Ejercicio 2.2: Verificación del Túnel

```
# Verifica que el puerto está escuchando
ss -tlnp | grep 9090

# Deberías ver algo como:
# LISTEN 0  128  127.0.0.1:9090  0.0.0.0:*  users:(("ssh",pid=1234,fd=3))

# Prueba el servicio
curl -v http://localhost:9090

# ¿Qué información ves en la respuesta?
# ____________________________________________
```

Ejercicio 2.3: Acceso vía Navegador

```
# Si tienes navegador gráfico en Kali:
firefox http://localhost:9090 &

# O usa lynx/links en modo texto:
lynx http://localhost:9090
```

Ejercicio 2.4: Múltiples Puertos

```
# Crea un segundo túnel a otro puerto (si existe)
ssh -i ~/.ssh/id_rsa_lab \
    -L 9095:localhost:22 \
   -N -f studentXX@192.168.99.50

# Ahora podrías hacer SSH al servidor a través del túnel:
ssh -p 9095 localhost
```

```
# ¡Cuidado! Esto crea un bucle SSH. ¿Por qué?
# ____________________________________________
```

Ejercicio 2.5: Túnel con Script

```
# Usa el script automatizado
./ssh_lab.sh local

# Verifica estado
./ssh_lab.sh status
```

4.4 Caso de Uso Real

Escenario: Una base de datos MySQL (puerto 3306) está en el servidor Ubuntu, pero solo escucha en `127.0.0.1` (no accesible desde fuera).

```
# Solución: Túnel SSH
ssh -L 3307:localhost:3306 studentXX@192.168.99.50

# En tu Kali, conecta a:
mysql -h 127.0.0.1 -P 3307 -u usuario -p
```

4.5 Preguntas

1. ¿Qué diferencia hay entre `localhost:9090` en tu Kali y `localhost:8080` en el servidor?
   Respuesta: **********_____**********

2. Si el servidor tiene 2 interfaces de red (eth0, eth1), ¿puedes acceder a servicios en eth1 desde tu Kali usando -L?
   Respuesta: **********_____**********

3. ¿Por qué el puerto local debe ser diferente al remoto?
   Respuesta: **********_____**********

---

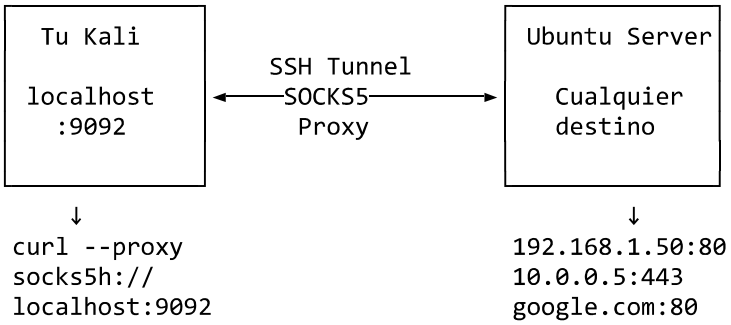
5. Práctica 3: Dynamic Port Forwarding (-D) - SOCKS Proxy

Duración: 60 minutos
Dificultad: ⭐⭐⭐ Intermedia-Avanzada

5.1 Objetivos

- Configurar un proxy SOCKS5 dinámico
- Entender la diferencia entre -L y -D
- Navegar "desde dentro" de la red del servidor

5.2 Teoría: ¿Qué es -D?



Diferencia clave:

- **-L**: Destino fijo y predefinido
- **-D**: Destino dinámico, decidido en tiempo de ejecución

5.3 Ejercicios

Ejercicio 3.1: Crear Proxy SOCKS

```
# Establece el túnel dinámico
ssh -i ~/.ssh/id_rsa_lab \
    -D 9092 \
   -N -f studentXX@192.168.99.50

# Verifica
ss -tlnp | grep 9092
```

Ejercicio 3.2: Prueba con curl

```
# Accede al servicio web de la VM vía proxy
curl --proxy socks5h://localhost:9092 http://localhost:8080

# Compara con el túnel -L:
curl http://localhost:9090

# ¿Obtienes el mismo resultado?
# ____________________________________________
```

Ejercicio 3.3: Resolución DNS Remota

```
# socks5h:// resuelve DNS desde el servidor
# socks5:// resuelve DNS localmente

# Prueba ambos:
curl --proxy socks5h://localhost:9092 http://google.com
curl --proxy socks5://localhost:9092 http://google.com

# ¿Hay diferencia? ¿Cuándo importa?
# ____________________________________________
```

Ejercicio 3.4: Configuración en Navegador

```
# Firefox/Chrome manual:
# 1. Abre Firefox
# 2. Preferencias → Configuración de red
# 3. Configuración manual de proxy:
#    SOCKS Host: 127.0.0.1
#    Puerto: 9092
#    Versión: SOCKS5
# 4. Marca "Proxy DNS cuando se use SOCKS v5"

# 5. Navega a http://192.168.99.50:8080
#    (la IP del servidor, no localhost)

# 6. ¿Funciona? ¿Por qué?
# ____________________________________________
```

Ejercicio 3.5: proxychains (Herramienta Avanzada)

```
# Instala proxychains (si no está)
sudo apt update && sudo apt install -y proxychains4

# Configura
```

```
echo "socks5 127.0.0.1 9092" | sudo tee -a /etc/proxychains4.conf

# Usa cualquier herramienta a través del proxy:
proxychains4 curl http://localhost:8080
proxychains4 nmap -sT 192.168.1.0/24
proxychains4 ssh studentXX@192.168.1.50

# ⚠️ Nota: nmap puede ser lento vía proxy
```

Ejercicio 3.6: Script Automatizado

```
./ssh_lab.sh dynamic
./ssh_lab.sh status
```

5.4 Caso de Uso Real

Escenario: Eres pentester y necesitas acceder a una red interna (10.0.0.0/24) que solo es visible desde el servidor Ubuntu comprometido.

```
# 1. Crea proxy SOCKS
ssh -D 9092 studentXX@192.168.99.50

# 2. Escanea la red interna
proxychains4 nmap -sT -Pn 10.0.0.0/24

# 3. Accede a servicios descubiertos
proxychains4 curl http://10.0.0.5:80
```

5.5 Comparativa: -L vs -D

Característica	-L (Local)	-D (Dynamic)
Destino	Fijo, predefinido	Dinámico, cualquier IP
Configuración	Simple	Requiere app con soporte proxy
Casos de uso	1 servicio específico	Múltiples servicios/red completa
Overhead	Bajo	Medio (SOCKS protocol)
Flexibilidad	Baja	Alta

5.6 Preguntas

1. ¿Cuándo usarías -D en lugar de -L?
   Respuesta: **********_____**********

2. ¿Por qué es importante `socks5h://` en lugar de `socks5://`?
   Respuesta: **********_____**********

3. Si cierras la sesión SSH, ¿qué pasa con las conexiones activas a través del proxy?
   Respuesta: **********_____**********

---

6. Práctica 4: Remote Port Forwarding (-R)

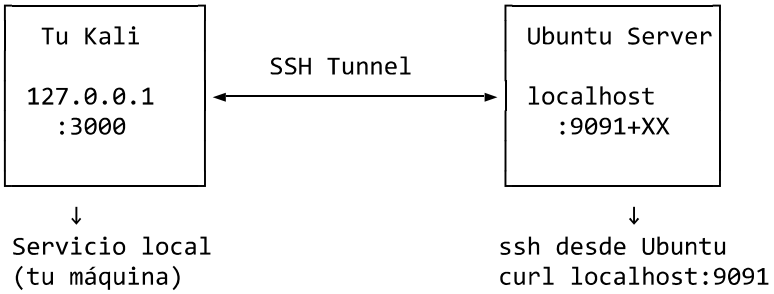
Duración: 50 minutos
Dificultad: ⭐⭐⭐ Intermedia-Avanzada

6.1 Objetivos

- Comprender el concepto de Remote Port Forwarding

- Exponer servicios locales a un servidor remoto
- Entender casos de uso en desarrollo y soporte

6.2 Teoría: ¿Qué es -R?



Dirección inversa:

- **-L**: Local → Remoto (accedes TÚ al servidor)
- **-R**: Remoto → Local (el servidor accede a TI)

6.3 Ejercicios

Ejercicio 4.1: Servicio Local de Demo

```
# Crea un servicio web simple en tu Kali
mkdir -p /tmp/mi_servicio
echo "<h1>Servicio de Alumno XX</h1><p>$(date)</p>" > /tmp/mi_servicio/index.html

cd /tmp/mi_servicio
python3 -m http.server 3000 --bind 127.0.0.1 &

# Verifica
curl http://localhost:3000
```

Ejercicio 4.2: Túnel Remote Manual

```
# Establece el túnel -R
# NOTA: El puerto remoto debe ser único por alumno
# student01 → 9091, student02 → 9092, etc.
PUERTO_REMOTO=$((9090 + XX))

ssh -i ~/.ssh/id_rsa_lab \
    -R ${PUERTO_REMOTO}:localhost:3000 \
   -N -f studentXX@192.168.99.50

# Verifica DESDE el servidor Ubuntu:
ssh studentXX@192.168.99.50 "ss -tln | grep $PUERTO_REMOTO"
```

Ejercicio 4.3: Acceso desde el Servidor

```
# Conéctate al servidor y prueba el túnel
ssh studentXX@192.168.99.50

# Dentro del servidor:
curl http://localhost:9091  # (cambia 9091 por tu puerto)

# Deberías ver el HTML de tu servicio local en Kali

# Sal
exit
```

Ejercicio 4.4: GatewayPorts (Avanzado)

```
# Por defecto, -R solo escucha en 127.0.0.1 del servidor
# Para que otras máquinas de la red accedan:

ssh -i ~/.ssh/id_rsa_lab \
    -o GatewayPorts=yes \
    -R 0.0.0.0:9091:localhost:3000 \
   -N -f studentXX@192.168.99.50

# Ahora cualquier máquina puede:
# curl http://192.168.99.50:9091

# ⚠️ Riesgo de seguridad: ¿Por qué?
# ____________________________________________
```

Ejercicio 4.5: Script Automatizado

```
./ssh_lab.sh remote
./ssh_lab.sh status
```

6.4 Caso de Uso Real

Escenario: Desarrollas una web app en localhost:3000 y quieres que un cliente la vea sin subirla a un servidor.

```
# 1. Túnel -R
ssh -R 8080:localhost:3000 tu_servidor.com

# 2. El cliente accede:
# http://tu_servidor.com:8080

# 3. ¡Los cambios en tu localhost se reflejan al instante!
```

6.5 Comparativa: -L vs -R

Característica	-L (Local)	-R (Remote)
Dirección	Tú → Servidor	Servidor → Tú
Quién inicia	Cliente SSH	Cliente SSH
Quién accede	Tú (desde local)	Otros (desde servidor)
Puerto escucha	Tu máquina	Servidor remoto
Caso típico	Acceder a BD interna	Demo desarrollo

6.6 Preguntas

1. ¿Por qué -R es útil para debuggear webhooks de GitHub/Stripe?
   Respuesta: **********_____**********

2. ¿Qué limitaciones tiene -R comparado con -L?
   Respuesta: **********_____**********

3. Si tu Kali cambia de IP (DHCP), ¿se rompe el túnel -R?
   Respuesta: **********_____**********

---

7. Práctica 5: Túneles Múltiples Simultáneos

Duración: 40 minutos
Dificultad: ⭐⭐ Intermedia

7.1 Objetivos

- Gestionar múltiples túneles concurrentes
- Entender interacciones entre túneles
- Optimizar recursos del sistema

7.2 Ejercicios

Ejercicio 5.1: Los 3 Túneles a la Vez

```
# Usa el modo persistente del script
./ssh_lab.sh all

# Verifica todos los puertos
ss -tlnp | grep -E '909[0-2]|3000'

# Deberías ver:
# 9090  (-L)
# 9092  (-D)
# 3000  (servicio local)
```

Ejercicio 5.2: Pruebas Concurrentes

```
# Terminal 1: Túnel -L
curl http://localhost:9090

# Terminal 2: Túnel -D
curl --proxy socks5h://localhost:9092 http://localhost:8080

# Terminal 3: Verifica túnel -R
ssh studentXX@192.168.99.50 "curl http://localhost:9091"

# ¿Funcionan los 3 simultáneamente?
# ____________________________________________
```

Ejercicio 5.3: Monitorización de Recursos

```
# En tu Kali, monitoriza:
watch -n 2 'ps aux | grep ssh | grep -v grep'

# Observa:
# - Número de procesos SSH
# - Uso de memoria
# - PIDs de cada túnel

# ¿Cuántos procesos SSH ves?
# ____________________________________________
```

Ejercicio 5.4: Cierre Selectivo

```
# Mata solo el túnel -L (puerto 9090)
pkill -f "ssh.*-L.*9090"

# Verifica
ss -tlnp | grep 9090  # Debería estar libre

# Los otros túneles siguen activos:
ss -tlnp | grep 9092  # Debería seguir activo

# Vuelve a crear solo -L
./ssh_lab.sh local
```

Ejercicio 5.5: Multiplexación SSH (Avanzado)

```
# En lugar de 3 conexiones SSH, usa 1 con multiplexación

# 1. Crea directorio de control
mkdir -p ~/.ssh/sockets

# 2. Añade a ~/.ssh/config:
cat >> ~/.ssh/config << 'EOF'
Host ubuntu-server
   HostName 192.168.99.50
    User studentXX
    ControlMaster auto
    ControlPath ~/.ssh/sockets/%r@%h-%p
    ControlPersist 600
EOF

# 3. Primera conexión (crea socket maestro)
ssh -i ~/.ssh/id_rsa_lab ubuntu-server

# 4. Los siguientes túneles reusan la misma conexión:
ssh -L 9090:localhost:8080 -N -f ubuntu-server
ssh -D 9092 -N -f ubuntu-server
ssh -R 9091:localhost:3000 -N -f ubuntu-server

# Verifica: solo 1 proceso SSH maestro
ps aux | grep ssh
```

7.3 Preguntas

1. ¿Cuántos procesos SSH se crean con 3 túneles manuales?
   Respuesta: **********_____**********

2. ¿Qué ventaja tiene ControlMaster?
   Respuesta: **********_____**********

3. ¿Puede un túnel -L y un túnel -D usar el mismo puerto local?
   Respuesta: **********_____**********

---

8. Práctica 6: Escenario Real - Acceso a Red Interna

Duración: 60 minutos
Dificultad: ⭐⭐⭐⭐ Avanzada

8.1 Objetivos

- Combinar múltiples técnicas de túnel
- Simular un escenario de pentesting real
- Acceder a recursos de red interna

8.2 Escenario

Contexto:
El servidor Ubuntu (192.168.99.50) tiene acceso a una red interna "oculta" (10.10.10.0/24) que no es reachable directamente desde tu Kali. Dentro de esa red hay:

- Un servidor web (10.10.10.50:80)
- Una base de datos MySQL (10.10.10.60:3306)
- Un servidor SSH (10.10.10.70:22)

Objetivo: Acceder a estos 3 servicios desde tu Kali.

8.3 Ejercicios

Ejercicio 6.1: Reconocimiento

```
# 1. Conéctate al servidor Ubuntu
ssh studentXX@192.168.99.50

# 2. Identifica interfaces de red
ip addr show

# 3. ¿Tiene acceso a 10.10.10.0/24?
ping -c 2 10.10.10.50

# 4. Escanea la red interna (desde Ubuntu)
sudo nmap -sT -Pn 10.10.10.0/24

# 5. Sal
exit
```

Ejercicio 6.2: Túnel -L a Servidor Web

```
# Crea túnel al web interno
ssh -i ~/.ssh/id_rsa_lab \
    -L 8080:10.10.10.50:80 \
   -N -f studentXX@192.168.99.50

# Accede desde tu Kali
curl http://localhost:8080

# O en navegador:
firefox http://localhost:8080 &
```

Ejercicio 6.3: Túnel a MySQL

```
# Crea túnel a BD
ssh -i ~/.ssh/id_rsa_lab \
    -L 3307:10.10.10.60:3306 \
   -N -f studentXX@192.168.99.50

# Conecta (si tienes mysql-client)
mysql -h 127.0.0.1 -P 3307 -u root -p

# O verifica con nmap
nmap -sV -p 3307 localhost
```

Ejercicio 6.4: Proxy SOCKS para Acceso Total

```
# En lugar de túneles individuales, usa -D
ssh -i ~/.ssh/id_rsa_lab \
    -D 9092 \
   -N -f studentXX@192.168.99.50

# Configura proxychains
echo "socks5 127.0.0.1 9092" | sudo tee -a /etc/proxychains4.conf

# Escanea la red interna DESDE tu Kali
proxychains4 nmap -sT -Pn 10.10.10.0/24

# Accede a cualquier servicio:
proxychains4 curl http://10.10.10.50:80
proxychains4 ssh root@10.10.10.70
```

Ejercicio 6.5: Túnel en Cascada (Avanzado)

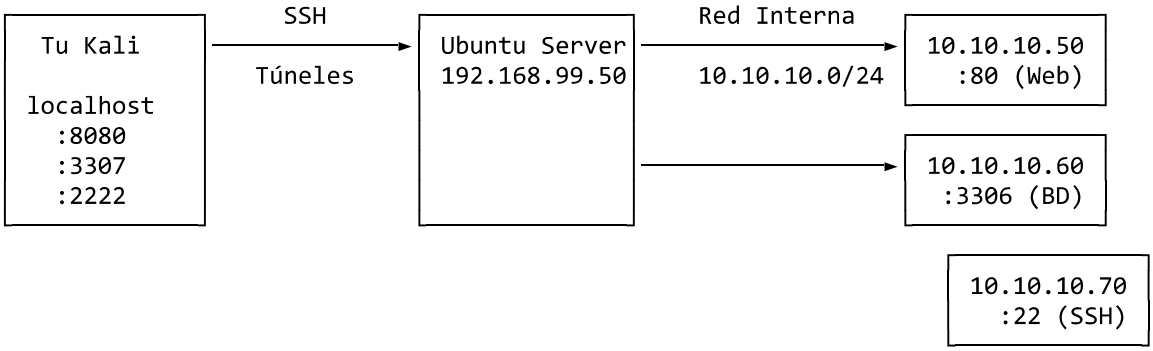
```
# Si 10.10.10.70 tiene SSH, crea un túnel en cascada:

# 1. Primero al servidor Ubuntu
ssh -i ~/.ssh/id_rsa_lab \
    -L 2222:10.10.10.70:22 \
   -N -f studentXX@192.168.99.50

# 2. Ahora conecta al SSH interno
ssh -p 2222 root@localhost

# ¡Estás en el servidor interno!
```

8.4 Diagrama del Escenario



8.5 Preguntas

1. ¿Qué método es más eficiente: múltiples -L o un -D con proxychains?
   Respuesta: **********_____**********

2. ¿Puedes crear un túnel -R desde el servidor interno hacia tu Kali?
   Respuesta: **********_____**********

3. Si la red interna usa IPs privadas no enrutables, ¿cómo llega el tráfico de vuelta a tu Kali?
   Respuesta: **********_____**********

---

9. Práctica 7: Seguridad y Auditoría de Túneles

Duración: 40 minutos
Dificultad: ⭐⭐⭐ Intermedia-Avanzada

9.1 Objetivos

- Identificar riesgos de seguridad en túneles SSH
- Auditar túneles activos
- Implementar buenas prácticas

9.2 Ejercicios

Ejercicio 7.1: Auditoría de Túneles Propios

```
# Lista todos tus túneles SSH
ps aux | grep "ssh.*-[LDR]"

# Para cada túnel, identifica:
# - PID
# - Tipo de túnel (-L, -D, -R)
# - Puertos involucrados
```

```
# - Usuario remoto

# Ejemplo de salida:
# kali  1234  0.0  0.1  ssh -L 9090:localhost:8080 -N -f student01@192.168.99.50
#                        ↑    ↑                ↑
#                       PID  Tipo            Destino
```

Ejercicio 7.2: Puertos Escuchando

```
# Verifica qué puertos están abiertos en tu Kali
ss -tlnp | grep LISTEN

# ¿Cuáles son de túneles SSH?
ss -tlnp | grep ssh

# ¿Alguno es accesible desde otras interfaces (0.0.0.0)?
ss -tlnp | grep '0.0.0.0'

# Riesgo: Si un puerto escucha en 0.0.0.0, cualquier máquina de la red puede acceder
```

Ejercicio 7.3: Túneles No Autorizados (Simulación)

```
# Como profesor, en el servidor Ubuntu:
sudo netstat -tlnp | grep sshd

# ¿Puedes identificar qué usuario tiene qué túnel?
sudo ps aux | grep sshd

# Ver logs de autenticación
sudo tail -50 /var/log/auth.log | grep student

# ¿Qué información revela?
# ____________________________________________
```

Ejercicio 7.4: Restricciones en sshd_config

```
# En el servidor Ubuntu (como root), edita:
sudo nano /etc/ssh/sshd_config

# Añade restricciones por usuario:
Match User student*
    # Solo permitir ciertos tipos de túnel
    AllowTcpForwarding local  # Solo -L, no -R ni -D
    # O:
    # AllowTcpForwarding no    # Deshabilitar todos los túneles

    # Limitar puertos remotos (-R)
    PermitOpen localhost:8080 localhost:3306

    # Forzar comando (evita shell interactiva)
    # ForceCommand echo "Solo túneles permitidos"

# Reinicia SSH
sudo systemctl restart ssh

# Prueba desde tu Kali:
ssh -D 9092 studentXX@192.168.99.50
# ¿Funciona? ¿Qué error da?
# ____________________________________________
```

Ejercicio 7.5: Túneles Cifrados - Verificación

```
# Verifica que el tráfico está cifrado
sudo tcpdump -i any -n port 22 -X | head -50
```

```
# ¿Ves datos legibles o cifrados?
# ____________________________________________

# Compara con HTTP sin cifrar:
tcpdump -i any -n port 80 -X | head -50
# ¿Ves diferencias?
```

Ejercicio 7.6: Buenas Prácticas

```
# 1. Usa claves con passphrase
ssh-keygen -p -f ~/.ssh/id_rsa_lab

# 2. Limita tiempo de vida del túnel
ssh -o ServerAliveInterval=60 -o ServerAliveCountMax=5 \
   -L 9090:localhost:8080 -N -f studentXX@192.168.99.50

# 3. Usa puertos no estándar (seguridad por oscuridad)
ssh -L 54321:localhost:8080 -N -f studentXX@192.168.99.50

# 4. Revoca acceso eliminando clave del servidor
ssh studentXX@192.168.99.50 "sed -i '/tu_clave/d' ~/.ssh/authorized_keys"
```

9.3 Checklist de Seguridad

Antes de crear un túnel:

- ☐ ¿El puerto local está libre?
- ☐ ¿Necesito realmente este túnel?
- ☐ ¿Estoy exponiendo datos sensibles?
- ☐ ¿La clave SSH tiene passphrase?

Después de crear un túnel:

- ☐ ¿Verifiqué que solo escucha en 127.0.0.1 (no 0.0.0.0)?
- ☐ ¿Documenté el propósito del túnel?
- ☐ ¿Sé cómo cerrarlo?

Al finalizar la práctica:

- ☐ Cerré todos los túneles
- ☐ Verifiqué con `ss -tlnp` que no quedan puertos abiertos
- ☐ Eliminé claves temporales si las creé

9.4 Preguntas

1. ¿Por qué es peligroso `GatewayPorts=yes` en producción?
   Respuesta: **********_____**********

2. ¿Puede un atacante interceptar el tráfico de un túnel SSH?
   Respuesta: **********_____**********

3. ¿Qué información revela `ps aux` sobre tus túneles?
   Respuesta: **********_____**********

Anexo A: Comandos de Referencia

A.1 Comandos Esenciales

```
# Local Port Forwarding (-L)
ssh -L [puerto_local]:[host_remoto]:[puerto_remoto] usuario@servidor

# Dynamic Port Forwarding (-D)
ssh -D [puerto_local] usuario@servidor

# Remote Port Forwarding (-R)
ssh -R [puerto_remoto]:localhost:[puerto_local] usuario@servidor

# Combinar opciones comunes
ssh -i ~/.ssh/id_rsa -o StrictHostKeyChecking=no -N -f usuario@servidor

# Ver túneles activos
ps aux | grep "ssh.*-[LDR]"
ss -tlnp | grep ssh

# Cerrar túnel específico
pkill -f "ssh.*-L.*9090"

# Ver logs de SSH
tail -f /var/log/auth.log  # En servidor
journalctl -f -u ssh       # En systemd
```

A.2 Script Rápido de Diagnóstico

```
#!/bin/bash
echo "🔍 Diagnóstico rápido:"
echo "1. Conectividad: $(nc -zv 192.168.99.50 22 &>/dev/null && echo '✓ OK' || echo '✗ FAIL')"
echo "2. Puertos libres: $(ss -tln | grep -E '909[0-2]' | wc -l) ocupados"
echo "3. Clave SSH: $([ -f ~/.ssh/id_rsa_lab ] && echo '✓ Existe' || echo '✗ No existe')"
echo "4. Túneles activos: $(ps aux | grep -c '[s]sh.*-[LDR]')"
```

Anexo B: Solución de Problemas

B.1 Problemas Comunes

Síntoma	Causa Probable	Solución
`Permission denied (publickey,password)`	Clave no copiada o password incorrecto	`ssh-copy-id` o verifica password
`Address already in use`	Puerto ocupado por otro proceso	`pkill -f ssh` o cambia puerto
`Connection refused`	SSH no corre en servidor o firewall	Verifica `systemctl status ssh`
`channel_setup_fwd_listener_tcpip`	GatewayPorts=no en servidor para -R	Contacta al profesor
Túnel se cierra solo	Timeout o pérdida de conexión	Usa `-o ServerAliveInterval=60`

B.2 Comandos de Debugging

```
# Modo verbose (muestra todo el proceso)
ssh -v -L 9090:localhost:8080 studentXX@192.168.99.50

# Debugging extremo
ssh -vvv studentXX@192.168.99.50

# Verifica configuración SSH
ssh -G studentXX@192.168.99.50 | grep -i forward

# Test de puerto sin SSH
nc -zv 192.168.99.50 22
telnet 192.168.99.50 22
```

B.3 Contacto de Soporte

Si encuentras un error no listado:

1. Ejecuta `./ssh_lab.sh check` y guarda la salida
2. Ejecuta `ps aux | grep ssh` y guarda la salida
3. Ejecuta `ss -tlnp | grep -E '909|3000'` y guarda la salida
4. Envía estas 3 salidas al profesor

Evaluación Final

Criterios de Aprobación

- ✅ Completa al menos 5 de 7 prácticas
- ✅ Responde correctamente el 70% de las preguntas de reflexión
- ✅ Demuestra funcionamiento de los 3 tipos de túneles
- ✅ Entrega informe con capturas de pantalla